Introduction to Algorithms

Levon Prazyan

Course Overview

- **Introduction**
- **Sorting and Searching**
- **Arrays and Lists**
- **Stacks and Queues, Double Ended Queue**
- **Heaps**
- **Trees**
- **Hash Tables**
- **Dynamic Programming**
- **Greedy Algorithms**
- **Graph Algorithms**
- **Network Flow**
- **NP-Completeness**

Dynmamic Programming

Richard Bellman

Charles Erwin Wilson (Engine Charlie)

Dynamic Programming (DP)

Dynamic programming is a technique used to

1. solve complex problems by breaking them down into simpler subproblems and
2. storing their solutions.

$$* DP \approx subproblems + "reuse“$$

We typically apply dynamic programming to ***optimization problems***.

Key Concepts of DP

1. Optimal Substructure:
   - A problem exhibits optimal substructure if an optimal solution to the problem can be constructed efficiently from optimal solutions of its subproblems. This property is fundamental for employing dynamic programming.
2. Overlapping Subproblems:
   - Dynamic programming is effective for problems with overlapping subproblems, meaning the same subproblems are solved multiple times. By solving each subproblem only once and storing the results, DP reduces the total number of computations.

Dynamic Programming Approaches

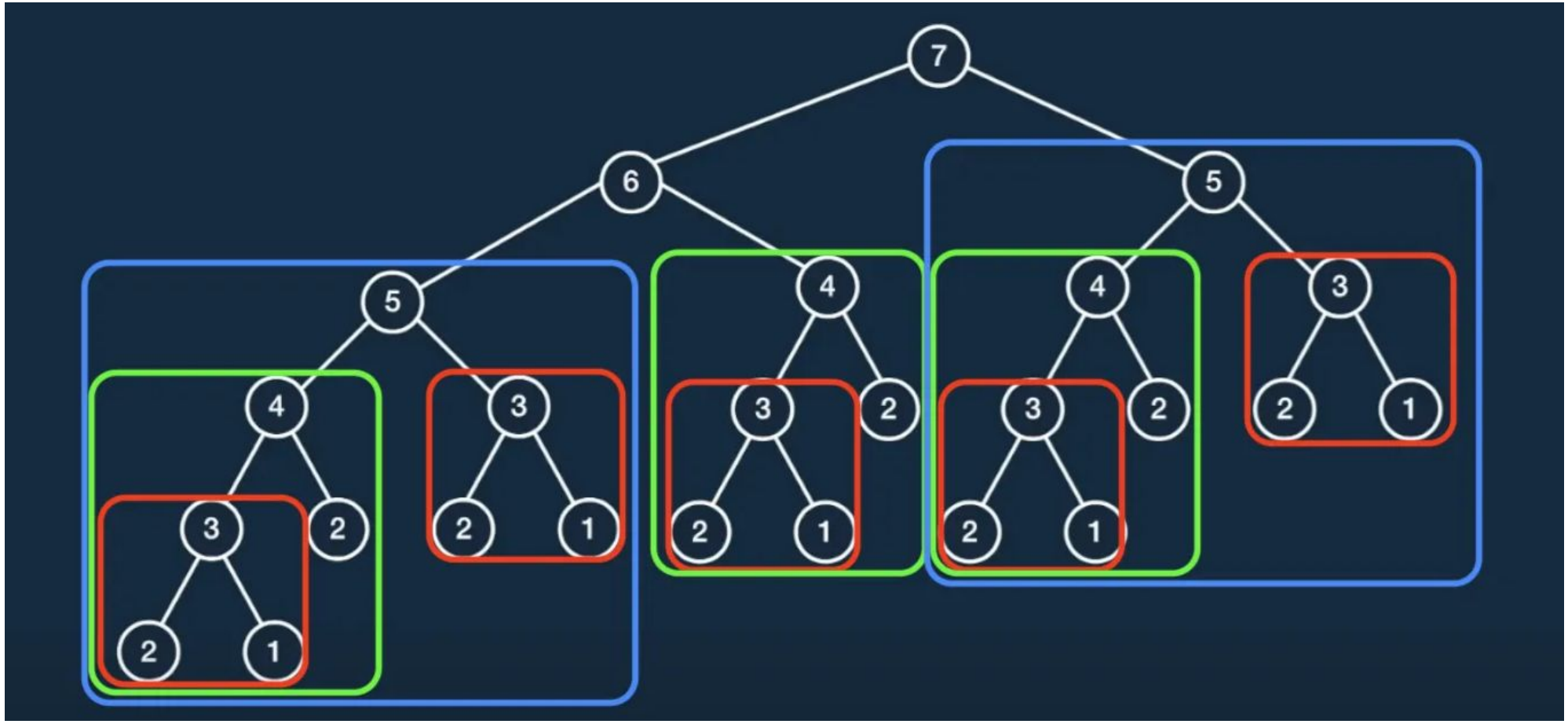
1. **Memoization**: top-down approach.
   - You recursively solve the subproblems and store their results in a data structure.
   - If you encounter the same subproblem again, you use the stored result instead of recomputing it.
2. **Tabulation**: bottom-up approach.
   - You iteratively solve all subproblems starting from the smallest one, and
   - build up to the solution of the entire problem by storing intermediate results.

DP Problem: Fibonacci Number (1)

```
namespace naive {

int fib(int n)
{
    if (n <= 1) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

} // naive namespace
```

- Time Complexity: $O(2^n)$
- Memory usage: $O(1)$

DP Problem: Fibonacci Number (2)



DP Problem: Fibonacci Number (3)

```
namespace memoization {

int fibHelper(int n, std::vector<int>& mem)
{
    if (n <= 1) {
        return n;
    }
    if (mem[n] != -1) {
        return mem[n];
    }
    mem[n] = fibHelper(n-1, mem) + fibHelper(n-2, mem);
    return mem[n];
}

int fib(int n)
{
    std::vector<int> mem(n+1, -1);
    return fibHelper(n, mem);
}

} // memoization namespace
```

- Time Complexity: $O(n)$
- Memory usage: $O(n)$

DP Problem: Fibonacci Number (4)

```
namespace tabulation {

int fib(int n)
{
    if (n <= 1) {
        return n;
    }
    std::vector<int> mem(n + 1, -1);
    mem[0] = 0;
    mem[1] = 1;
    for (int i = 2; i <= n; ++i) {
        mem[i] = mem[i - 1] + mem[i -2];
    }
    return mem[n];
}

} // tabulation namespace
```

- Time Complexity: $O(n)$
- Memory usage: $O(n)$

DP Problem: Fibonacci Number (5)

-
- $\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n} = \begin{bmatrix} F(n+1) & F(n) \\ F(n) & F(n\text{-}1) \end{bmatrix}$

- $\begin{bmatrix} F(2n+1) & F(2n) \\ F(2n) & F(2n-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{2n} = \left(\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n}\right)^{2} = \left(\begin{bmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{bmatrix}\right)^{2} =$

  $$\begin{bmatrix} F(n+1)^2 + F(n)^2 & F(n+1)F(n) + F(n)F(n-1) \\ F(n+1)F(n) + F(n)F(n-1) & F(n)^2 + F(n-1)^2 \end{bmatrix}$$

- $F(2n) = F(n)(2F(n+1) - F(n)); \quad F(2n-1) = F(n)^2 + F(n-1)^2$

Tabulation vs Memoization (1)

Tabulation

- Bottom-up approach
- Stores the results of subproblems in a table
- Iterative implementation
- Entries are filled in a bottom-up manner from the smallest size to the final size.

Memoization

- Top-down approach
- Stores the results of function calls in a table.
- Recursive implementation
- Entries are filled when needed.

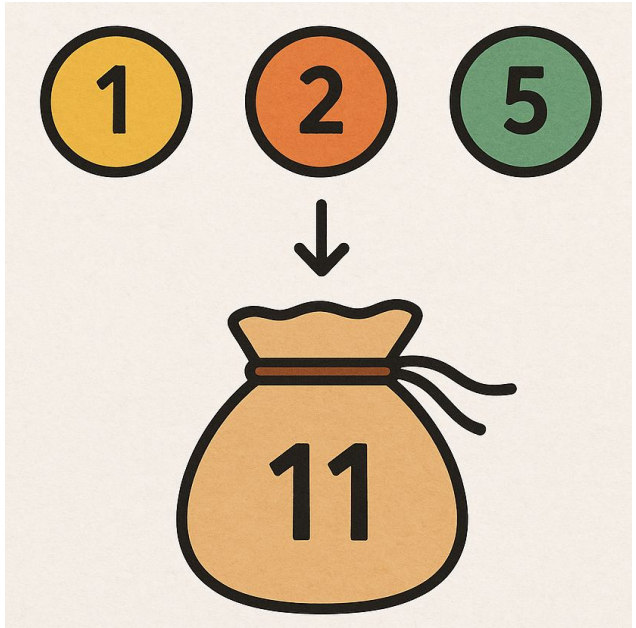
Tabulation vs Memoization (2)

	Tabulation	Memoization
State	State transition relation is difficult to think.	State Transition relation is easy to think.
Code	Code gets complicated when a lot of conditions are required.	Code is easy to write by modifying the underlying recursive solution.
Speed	Fast - no recursion calls.	Slow due to a lot of recursive calls.
Subproblem Solving	If all subproblems must be solved at least once, a bottom-up dynamic programming algorithm definitely outperforms a top-down memoized algorithm by a constant factor	If some subproblems in the subproblem space need not be solved at all, the memoized solution has the advantage of solving only those subproblems that are definitely required
Table Entries	In the Tabulated version, starting from the first entry, all entries are filled one by one	Unlike the Tabulated version, all entries of the lookup table are not necessarily filled in Memoized version. The table is filled on demand.

DP: Coin Change Problem (1)

Given a set of coin denominations and a target amount, determine the fewest number of coins needed to make up that amount.

If there is no combination that can achieve the amount, return -1

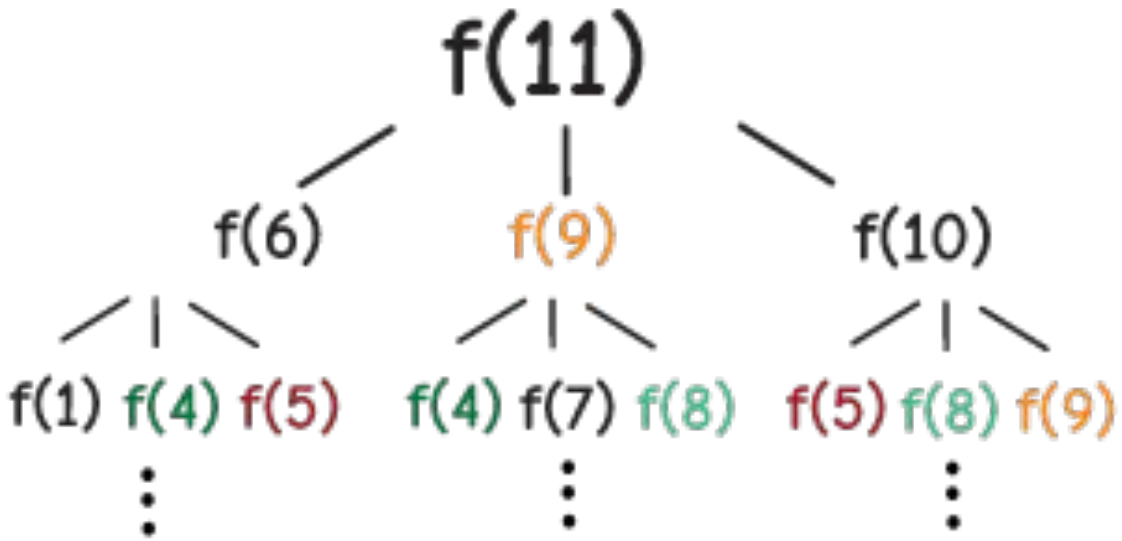


DP: Coin Change Problem (2)

Memoization

$$f(n) = \min \begin{pmatrix} f(n\text{-}coins[1]) \\ f(n\text{-}coins[2]) \\ \vdots \\ f(n\text{-}coins[j]) \end{pmatrix}$$

Example: coins = [1, 2, 5] , amount = 11



DP: Coin Change Problem (3)

```
namespace memoization {

int coinChangeHelper(const std::vector<int>& coins, int amount, std::vector<int>& memo)
{
   if (amount < 0) { return -1; }
   if (amount == 0) { return 0; }
   if (memo[amount] != -1) { return memo[amount]; }

   int minCoins = std::numeric_limits<int>::max();
   for (int coin : coins) {
      int currCoins = coinChangeHelper(coins, amount - coin, memo) + 1;
      if (currCoins > 0 && currCoins < minCoins) {
         minCoins = currCoins;
      }
   }
   memo[amount] = (minCoins == std::numeric_limits<int>::max()) ? -1 : minCoins;
   return memo[amount];
}

int coinChange(const std::vector<int>& coins, int amount)
{
   std::vector<int> memo(amount + 1, -1);
   return coinChangeHelper(coins, amount, memo);
}

} // memoization namespace
```

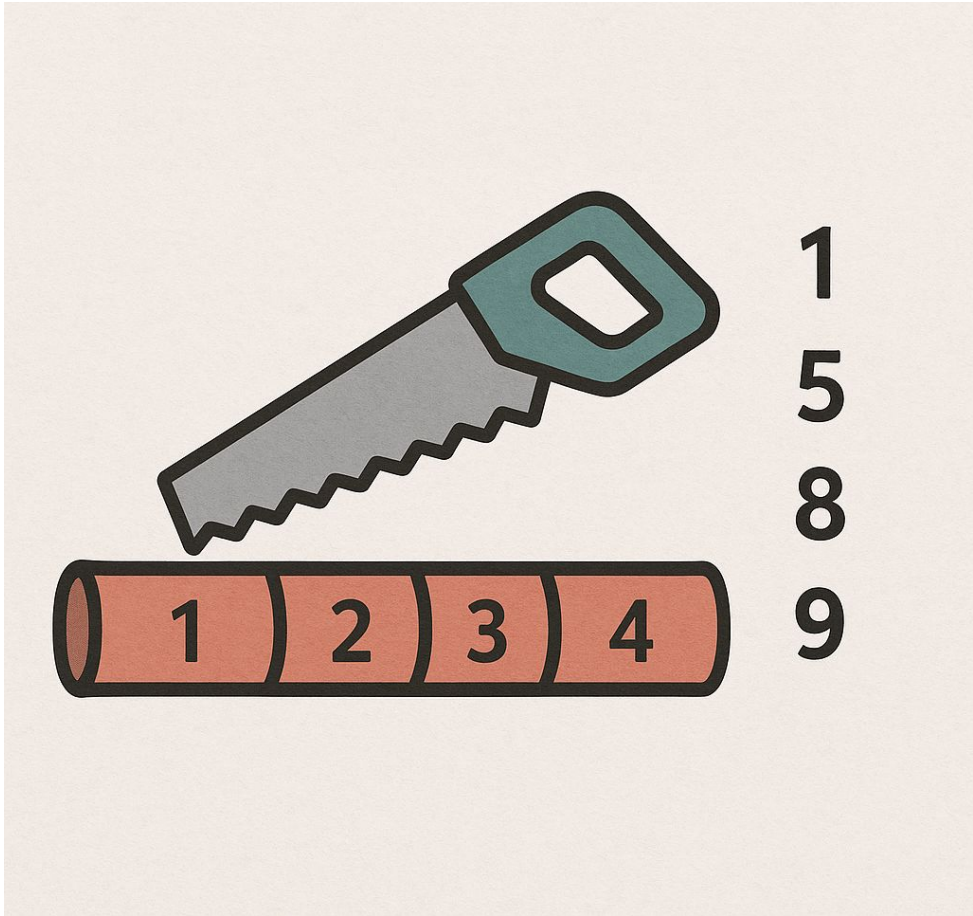
DP: Coin Change Problem (4)

```cpp
namespace tabulation {

int coinChange(const std::vector<int>& coins, int amount)
{
   std::vector<int> numCoins(amount + 1, std::numeric_limits<int>::max());
   numCoins[0] = 0;

   for (int i = 1; i <= amount; ++i) {
      for (int coin : coins) {
         if (i - coin >= 0 && numCoins[i - coin] != std::numeric_limits<int>::max()) {
            numCoins[i] = std::min(numCoins[i], numCoins[i - coin] + 1);
         }
      }
   }
   return numCoins[amount] == std::numeric_limits<int>::max() ? -1 : numCoins[amount];
}

} // tabulation namespace
```

Rod Cutting Problem (1)

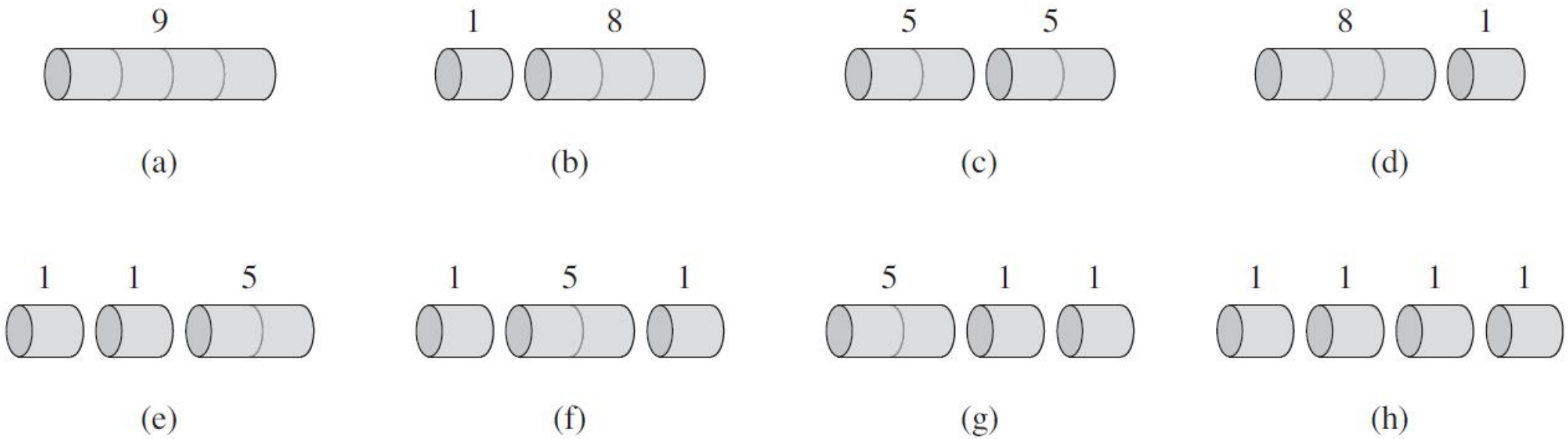
Given

- a rod of length n and
- an array of prices where i-th entry represents a price of a rod of length $i + 1$.
- Find the maximum revenue obtainable by cutting up the rod and selling the pieces.



Rod Cutting Problem (2)

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30



Rod Cutting Problem (2)

```
namespace naive {

int cutRod(int length, const std::vector<int>& prices)
{
    if (length <= 0) { return 0; }
    if (length + 1 > prices.size()) { return 0; }
    int max = std::numeric_limits<int>::min();
    for (int i = 1; i <= length; ++i) {
        max = std::max(max, prices[i] + cutRod(length - i, prices));
    }
    return max;
}

} // naive namespace
```

Rod Cutting Problem (3)

```
int cutRodHelper(int length, const std::vector<int>& prices, std::vector<int>& memo)
{
   if (length <= 0) { return 0; }
   if (length + 1 > prices.size()) { return 0; }

   if (memo[length] != std::numeric_limits<int>::min()) {
      return memo[length];
   }
   int max = std::numeric_limits<int>::min();
   for (int i = 1; i <= length; ++i) {
      max = std::max(max, prices[i] + cutRodHelper(length - i, prices, memo));
   }
   memo[length] = max;
   return max;
}

int cutRod(int length, const std::vector<int>& prices)
{
   std::vector<int> memo(length + 1, std::numeric_limits<int>::min());
   memo[0] = 0;
   return cutRodHelper(length, prices, memo);
}
```

Rod Cutting Problem (4)

```
int cutRod(int length, const std::vector<int>& prices)
{
   if (length + 1 > prices.size()) { return 0; }

   std::vector<int> dp(length + 1, 0);
   for (int i = 1; i <= length; ++i) {
      int max = prices[i];
      for (int j = 1; j <= i; ++j) {
         if (max < prices[j] + dp[i-j]) {
            max = prices[j] + dp[i-j];
         }
      }
      dp[i] = max;
   }
   return dp[length];
}
```

DP: Knapsack Problem (1)

Given a set of items, each with

- a weight and
- a value,

determine which items to include in a knapsack so that

- the total weight does not exceed a given capacity, and
- the total value is as large as possible.

Each item can be included at most once.

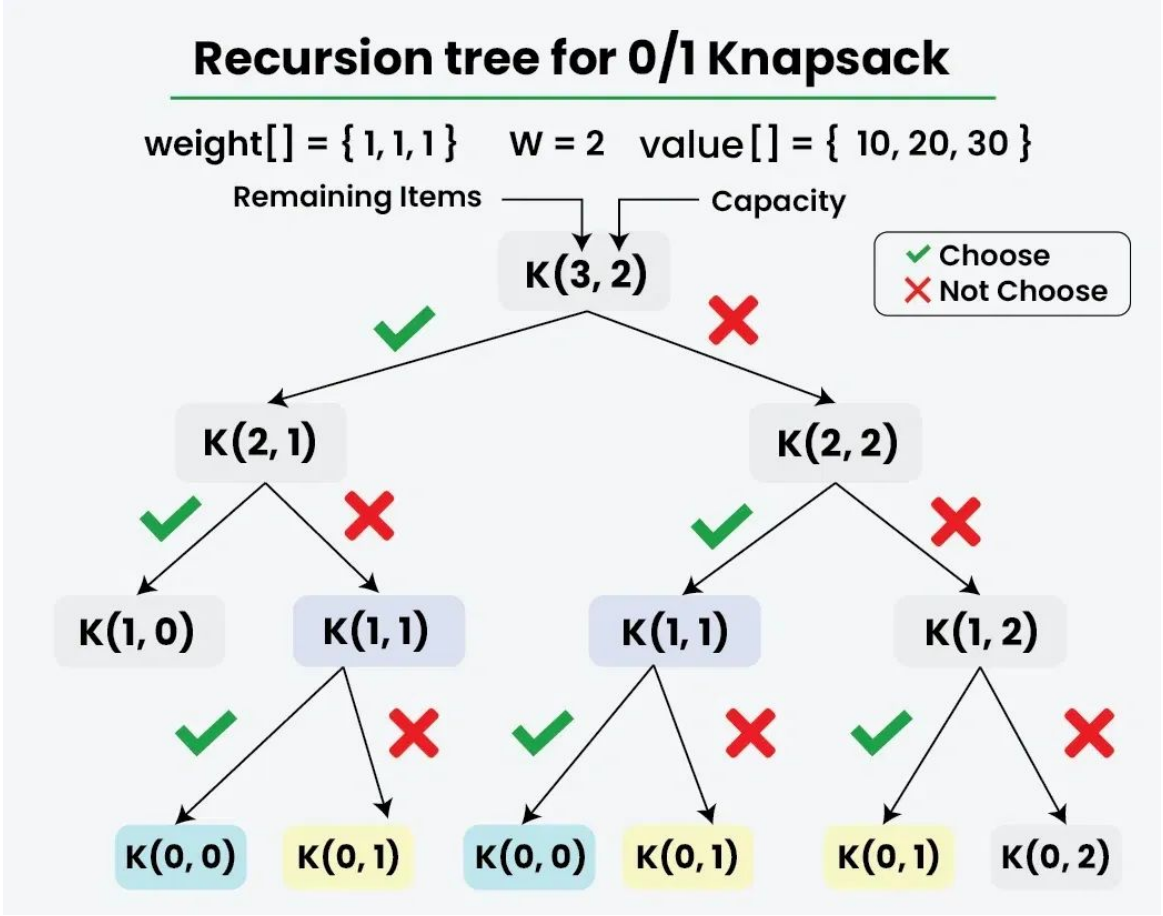
DP: Knapsack Problem (2)

```
namespace naive {

int knapsackHelper(int cap, const std::vector<int>& values, const std::vector<int>& weights, int itemId)
{
   if (itemId == 0 || cap == 0) { return 0; }

   int pick = 0;
   if (weights[itemId - 1] <= cap) {
      pick = values[itemId - 1] + knapsackHelper(cap - weights[itemId - 1], values, weights, itemId - 1);
   }
   int notPick = knapsackHelper(cap, values, weights, itemId - 1);
   return std::max(pick, notPick);
}

int knapsack(int cap, const std::vector<int> &values, const std::vector<int>& weights)
{
    int lastItemId = values.size();
    return knapsackHelper(cap, values, weights, lastItemId);
}

} // naive namespace
```

DP: Knapsack Problem (3)

```
namespace memoization {

int knapsackHelper(int cap, const std::vector<int>& values, const std::vector<int>& weights, int itemId,
                   std::vector<std::vector<int>>& memo)
{
    if (itemId == 0 || cap == 0) { return 0; }

    if(memo[itemId][cap] != -1) { return memo[itemId][cap]; }

    int pick = 0;
    if (weights[itemId - 1] <= cap) {
        pick = values[itemId - 1] + knapsackHelper(cap - weights[itemId - 1], values, weights, itemId - 1, memo);
    }
    int notPick = knapsackHelper(cap, values, weights, itemId - 1, memo);
    return memo[itemId][cap] = std::max(pick, notPick);
}

int knapsack(int cap, const std::vector<int>& values, const std::vector<int>& weights)
{
    int lastItemId = values.size();
    std::vector<std::vector<int>> memo(lastItemId + 1, std::vector<int>(cap + 1, -1));

    return knapsackHelper(cap, values, weights, lastItemId, memo);
}

} // memoization namespace
```

DP: Knapsack Problem (4)



DP: Knapsack Problem (5)

```cpp
int knapsack(int cap, const std::vector<int>& values, const std::vector<int>& weights)
{
    int n = weights.size();
    // Create a DP table initialized to zero
    std::vector<std::vector<int>> dp(n + 1, std::vector<int>(cap + 1, 0));

    for (int i = 1; i <= n; ++i) {
        for (int w = 0; w <= cap; ++w) {
            if (weights[i - 1] <= w) {
                dp[i][w] = std::max(dp[i - 1][w], dp[i - 1][w - weights[i - 1]] + values[i - 1]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][cap];
}
```

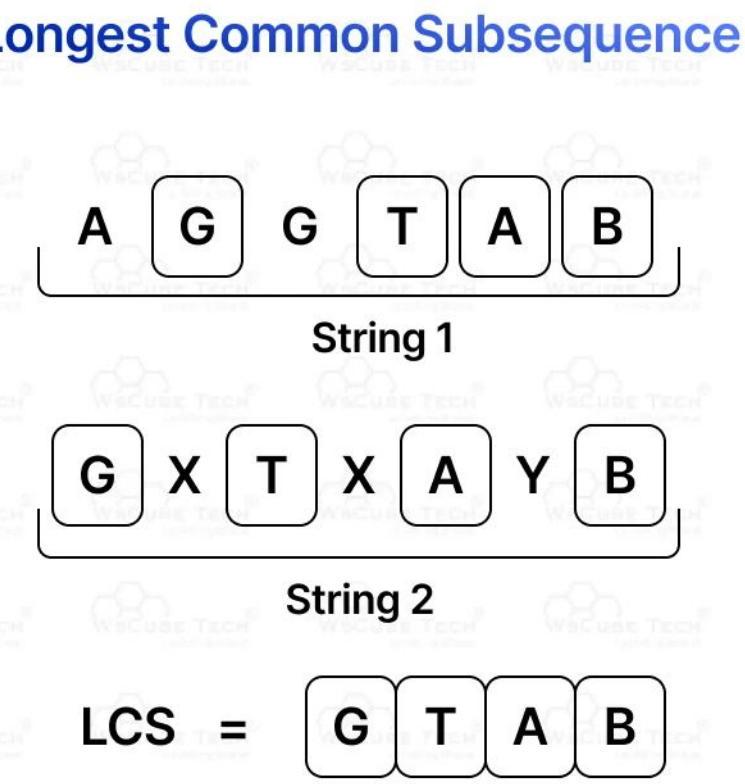
DP: Longest Common Subsequence (1)

- Given two sequences, such as strings X and Y, find the length of the longest subsequence that can appear in both X and Y.

Example:

Input: X = "ABCBDAB", Y = "BDCAB"

Output: LCS = "BCAB", with a length of 4



DP: Longest Common Subsequence (2)

```
namespace naive {

int longestCommonSubseq(const std::string& a, const std::string& b)
{
   if (a.size() == 0 || b.size() == 0) {
      return 0;
   }
   std::string a1 = a.substr(0, a.size()-1);
   std::string b1 = b.substr(0, b.size()-1);
   if (a.back() == b.back()) {
      return 1 + longestCommonSubseq(a1, b1);
   }
   return std::max(longestCommonSubseq(a1, b),
                   longestCommonSubseq(a , b1));
}

} // naive namespace
```

DP: Longest Common Subsequence (3)

```
int longestCommonSubseqHelper(const std::string& a, const std::string& b,
                              std::size_t aId, std::size_t bId,
                              std::vector<std::vector<int>>& memo)
{
   if (aId == 0 || bId == 0) { return 0; }
   if (memo[aId][bId] != -1) { return memo[aId][bId];}
   if (a[aId - 1] == b[bId - 1]) {
      memo[aId][bId] = 1 + longestCommonSubseqHelper(a, b, aId - 1, bId - 1, memo);
   } else {
      memo[aId][bId] = std::max(longestCommonSubseqHelper(a, b, aId - 1, bId    , memo),
                                longestCommonSubseqHelper(a, b, aId,     bId - 1, memo));
   }
   return memo[aId][bId];
}

int longestCommonSubseq(const std::string& a, const std::string& b)
{
   std::vector<std::vector<int>> memo(a.size() + 1, std::vector<int>(b.size() + 1, -1));
   return longestCommonSubseqHelper(a, b, a.size(), b.size(), memo);
}
```

DP: Longest Common Subsequence (4)

```
namespace tabulation {

int longestCommonSubseq(const std::string& a, const std::string& b)
{
    std::vector<std::vector<int>> dp(a.size() + 1, std::vector<int>(b.size() + 1, 0));
    for (int i = 1; i <= a.size(); ++i) {
        for (int j = 1; j <= b.size(); ++j) {
            if (a[i-1] == b[j-1]) {
                dp[i][j] = 1 + dp[i - 1][j - 1];
            } else {
                dp[i][j] = std::max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[a.size()][b.size()];
}

} // tabulation namespace
```

Thank You